



Laboratorio #4

Navegadores: Herramientas para desarrolladores

Herramienta para desarrolladores en navegadores

Permiten depurar, debuggear, ver más detalles del desarrollo, atributos de css, html, JavaScript, incluso evaluar la velocidad al evaluar llamadas y tiempos de respuestas por request, entre otras utilidades.

Existen tres distintas maneras de acceder:

- **Teclado:**

Presionando las teclas Ctrl+Mayús+I y/o F12 , excepto en macOS donde se utiliza Command+Option+I

- **Barra de menú:**

- **Chrome:** Más herramientas ➤ Herramientas del desarrollador
- **Firefox:** Menú ➤ Desarrollador web ➤ Alternar herramientas, o ➤ Herramientas ➤ Alternar herramientas del desarrollador web
- **Safari:** Desarrollador ➤ Mostrar el inspector web. Si no puedes ver el menú Desarrollador, ve a Safari ➤ Preferencias ➤ Avanzado y marca la casilla de verificación Mostrar menú desarrollador en la barra de menú.
- **Opera:** Desarrollador ➤ Herramientas para desarrolladores

- **Menú contextual:**

Manteniendo presionado / haciendo clic con el botón derecho en un elemento en una página web (Ctrl-clic en Mac) y elegir "Inspeccionar".

Ejercicios

a) Inspeccionar el HTML

1. Accede al aula virtual y descarga el comprimido Clase 12 - Laboratorio 4: Navegador.
2. Luego de descomprimirlo, abrir en Visual Studio Code los 3 archivos que se encuentran en el directorio que ha descargado.
3. Abre en el navegador el archivo "laboratorio4.html".
4. Pulse F12 para abrir las herramientas de desarrollador. En la pestaña de "inspector" hacer clic derecho sobre un nodo del DOM abre el menú contextual.

Las opciones disponibles en el menú varían según el navegador, pero en su mayoría, las más importantes son las mismas:

- **Eliminar nodo:** Elimina el elemento actual.

- **Editar como HTML:** Te permite cambiar el HTML y ver los resultados en tiempo real. Muy útil para depurar y probar.
 - **:hover/:active/:focus:** Obliga a que se activen los estados de los elementos, para que puedas ver cómo se vería su estilo.
 - **Copiar/Copiar como HTML:** Copie el HTML seleccionado actualmente.
5. Haga clic sobre la opción “editar como HTML” en el menú contextual y edite el contenido del nodo seleccionado.
 6. Verifique el archivo html en VSC.
 7. Refresque el navegador.
 8. ¿Qué sucedió?, ¿se han guardado los cambios? ¿son permanentes?.
 9. Haga clic derecho sobre la página (no sobre las herramientas de desarrollador) y seleccione la opción “Ver código fuente”, busque la palabra TABLE.
 10. Ahora revise el inspector de la página y ubique la misma etiqueta TABLE ¿Qué nota?.

b) Inspeccionar el DOM

1. Haz clic con el botón derecho en “Item 1” del html del laboratorio en “Parte 3: Explorar DOM”, a continuación selecciona “Inspeccionar”.
2. ¿Cuál considera es la diferencia entre ver el html en fuente y verlo en el inspector de DOM?

c) Editar el DOM:

1. Haz clic derecho en “Item 2” a continuación y selecciona Inspeccionar.
2. En el árbol del DOM, haz doble clic en “Item 2” y modifica su valor. En otras palabras, haz doble clic en el texto entre y . El texto se destaca en azul para indicar que está seleccionado.
3. Presione la tecla H para ocultar el nodo. También puedes hacer clic derecho en el nodo y usar la opción “Ocultar elemento”.

```
<li class="__web-inspector-hide-shortcut__"> == $0
  ::marker
  "Item 2"
</li>
```

d) Herramienta de Red y reglas de diseño DOM:

La carga del sitio web es decisiva en el posicionamiento SEO de cualquier página web. Una carga rápida reducirá la tasa de rebote, ayudará a mejorar el CRO (Conversion Rate Optimization); es decir, mejorará la conversión de visitantes en leads y, por supuesto, mejorará la indexación.

Para poner el Document Object Model a dieta debemos reducir el número de contenedores al máximo, cambiar la forma en la que construimos nuestros sitios webs. Par de prácticas sencillas que podemos llevar a cabo para reducirlo al máximo son:

- Prestar atención a todas las landings de una página.
- El uso de porcentajes para distribuir los contenidos dentro de un único contenedor es mucho mejor que crear varios contenedores dentro del mismo. Evitar las etiquetas <div> repetitivas puede ser determinante en estos casos.

1. Acceda a una página cualquiera y abra las herramientas de desarrollador.
2. Haga clic en la pestaña de Red o Network y chequee los requests y tiempos de respuestas.
3. ¿Qué opina de ésta herramienta? ¿Qué utilidad consideras tiene esta herramienta?

e) Inspeccionar CSS:

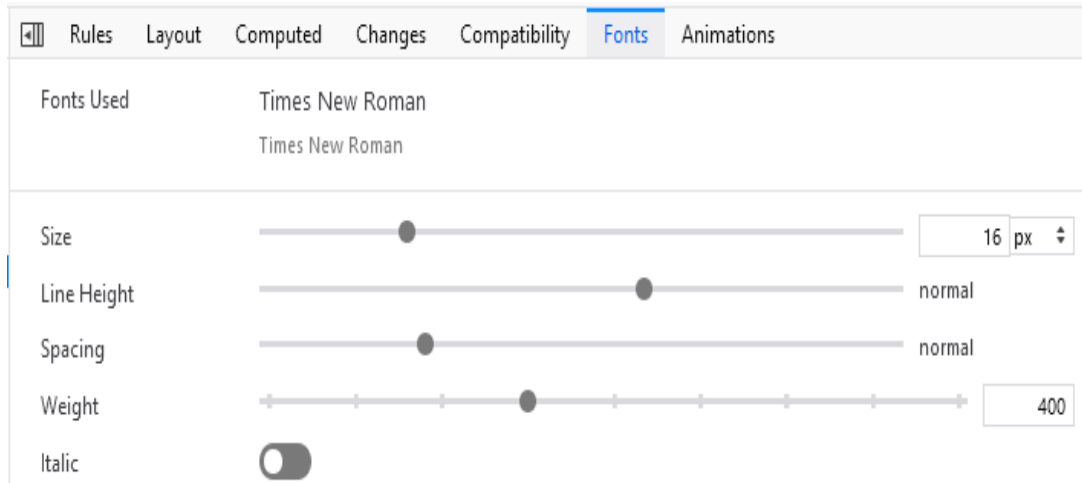
1. En la página del laboratorio selecciona alguno de los demos amarillo o rojo.

De manera predeterminada, el editor CSS muestra las reglas CSS aplicadas al elemento seleccionado actualmente.

Estas características son especialmente útiles:

- Las reglas aplicadas al elemento actual se muestran en orden de mayor a menor especificidad.
 - La pequeña flecha al lado de la abreviatura de cada propiedad muestra los nombres completos equivalentes de la propiedad.
 - Al hacer clic en el nombre o valor de una propiedad aparecerá un cuadro de texto, donde puedes ingresar un nuevo valor para obtener una vista previa en vivo de un cambio de estilo.
 - Junto a cada regla está el nombre del archivo y el número de línea en el que se define la regla. Al hacer clic en esta regla, las herramientas de desarrollo saltan para mostrarlas a su propia vista, donde generalmente puedes editar y guardar.
 - También puedes hacer clic en la llave de cierre de cualquier regla para que aparezca un cuadro de texto en una nueva línea, donde puedes escribir una declaración completamente nueva para tu página.
2. Haz clic en las casillas de verificación junto al tamaño de la fuente.
 3. Comparta con sus compañeros para que podría ser útil la opción de eliminar la declaración/atributo de un estilo css desde el navegador.
 4. Explorar las otras pestañas del inspector de CSS:
 - **Calculado:** Muestra los estilos calculados para el elemento seleccionado actualmente (los valores finales normalizados que aplica el navegador).

- **Diseño:** En Firefox, esta área incluye dos secciones:
- **Modelo de caja:** representa visualmente el modelo de caja del elemento actual, por lo que de un vistazo puedes identificar qué relleno, borde y margen se le aplica, y qué tan grande es su contenido.
- **Cuadrícula:** Si la página que estás inspeccionando utiliza Grid CSS, esta sección te permite ver los detalles de la cuadrícula.
- **Fuentes:** En Firefox, la pestaña Fuentes muestra los tipos de letra aplicados al elemento actual.



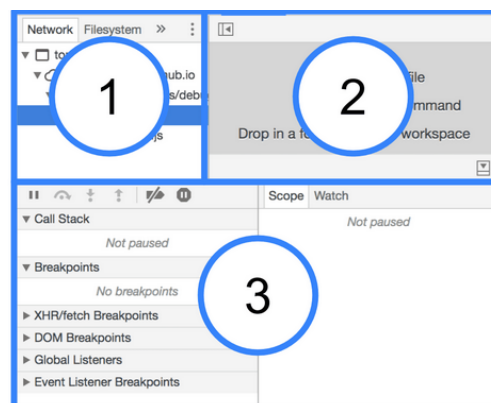
f) Crear el CSSOM y árbol de renderizado:

1. Dibuje el árbol DOM de la página del laboratorio.
2. Revise las reglas de estilo aplicadas a cada nodo y dibuje el árbol CSSOM usando de base el árbol DOM.
3. Una vez creado el árbol CSSOM, dibuje el árbol de renderizado de la página.

g) Inspeccionar JavaScript:

1. Trabajaremos con los archivos de "laboratorio4", abre el panel Console → Sources

La IU del panel Sources consta de 3 partes:



1. **Panel File Navigator** Aquí se muestran todos los archivos que solicita la página.
2. **El panel Code Editor** (Editor de código). Después de seleccionar un archivo en el panel File Navigator, su contenido se muestra aquí.
3. **El panel JavaScript Debugging** Diversas herramientas para inspeccionar el código JavaScript de la página. Si la ventana de Herramientas para desarrolladores es ancha, este panel se muestra a la derecha del panel Editor de código.
2. En el html de laboratorio 4, sección “Parte 4: Debugging JavaScript with Chrome DevTools”, se puede asumir que la suma es incorrecta, dado que los números se están concatenando, en lugar de sumarse. Por lo tanto, hagamos el ejercicio de pausar el código cerca del momento en que se ejecuta el objeto de escucha click.
3. En el panel JavaScript Debugging, haz clic en Event Listener Breakpoints para expandir la sección. Las Herramientas para desarrolladores muestran una lista de categorías de eventos expandibles, como Animación y Portapapeles.
4. Junto a la categoría de evento Mouse, haz clic en Expandir . Las Herramientas para desarrolladores muestran una lista de eventos del mouse, como click y mousedown. Cada evento tiene una casilla de verificación junto a él.
5. Marca la casilla de verificación de clic. Herramientas para desarrolladores ahora está configurado para pausar automáticamente cuando se ejecute cualquier objeto de escucha de eventos click.
6. En la demostración, vuelve a hacer clic en “SUMAR Numero 1 y Numero 2 (no concatenar)”. Herramientas para desarrolladores pausa la demostración y destaca una línea de código en el panel Sources. Las Herramientas para desarrolladores deben pausarse en esta línea de código:

```
function onClick() {...}
```

Si hiciste la pausa en una línea de código diferente, presiona Resume Script Execution  hasta que el elemento esté pausado en la línea correcta.

7. Recorre el código. En el panel Sources de Herramientas para desarrolladores, haz clic en Step into next function call (para chrome , en firefox ) para recorrer la ejecución de la función onClick(), una línea a la vez. Herramientas para desarrolladores destaca la siguiente línea de código:

```
if (inputsAreEmpty()) {...}
```

8. Haz clic en Step over next function call . Herramientas para desarrolladores ejecuta inputsAreEmpty() sin recorrerla. Observa cómo Herramientas para desarrolladores omiten algunas líneas de código. Esto se

debe a que `inputsAreEmpty()` se evaluó como falso, por lo que no se ejecutó el bloque de código de la sentencia `if`.

Esa es la idea básica de recorrer el código. Si observas el código de `get-started.js`, puedes ver que el error está probablemente en algún lugar de la función `updateLabel()`. En lugar de recorrer cada línea de código, puedes usar otro tipo de punto de interrupción para pausar el código más cerca de la ubicación probable del error.

9. Establece un punto de interrupción de línea de código. Para ello observa la última línea de código en `updateLabel()`:

```
label.textContent = addend1 + ' + ' + addend2 + ' = ' + sum;
```

A la izquierda del código, puedes ver el número de esta línea de código en particular. Las Herramientas para desarrolladores colocan un ícono azul sobre la línea al hacer clic sobre este número. Esto significa que hay un punto de interrupción de línea de código en esta línea. Herramientas para desarrolladores ahora siempre hace una pausa antes de que se ejecute esta línea de código.

10. Haz clic en Reanudar ejecución de la secuencia de comandos . La secuencia de comandos continúa ejecutándose, la Herramientas para desarrolladores muestra los valores de `addend1`, `addend2` y `sum` intercalados junto a sus declaraciones.
11. Verifica los valores de las variables. Los valores de `addend1`, `addend2` y `sum` están entre comillas, lo que significa que son cadenas de caracteres. Para recopilar más información sobre el error, las herramientas para desarrolladores proporcionan muchas herramientas para examinar valores de variables.
12. ¿Qué diferencia existe entre usar `console.log()` y breakpoints?

- h) Inspeccionar el panel de Permiso.

Quando el código está pausado en una línea de código, el panel Scope te muestra qué variables locales y globales están definidas actualmente, junto con el valor de cada variable. También muestra variables de cierre, cuando corresponde. Haz doble clic en el valor de una variable para editarlo.

Quando no estás en una línea de código, el panel Scope está vacío.

- i) Inspeccionar las expresiones supervisadas.

La pestaña Watch Expressions te permite supervisar los valores de las variables a lo largo del tiempo. Como su nombre lo implica, las expresiones supervisadas no se limitan a variables. Puedes almacenar cualquier expresión válida de JavaScript en una expresión supervisada. Probar ahora:

1. Haz clic en la pestaña Mirar.

2. Haz clic en Add Expression .
3. Tipo `typeof sum`.
4. Presione Intro. Herramientas para desarrolladores muestra `typeof sum`: "string". El valor que aparece a la derecha de los dos puntos es el resultado de la expresión supervisada.

j) Usar la consola.

1. Además de ver los mensajes de `console.log()`, también puedes usar la consola para evaluar sentencias arbitrarias de JavaScript. En términos de depuración, puedes usar Console para probar posibles correcciones de errores. Probar ahora:
2. Si no tienes abierto el panel lateral de la consola, presiona Escape para abrirlo. Se abrirá en la parte inferior de la ventana de Herramientas para desarrolladores.
3. En la consola, escribe `parseInt(addend1) + parseInt(addend2)`. Esta instrucción funciona porque el código está pausado en una línea en la que `addend1` y `addend2` están dentro del alcance.
4. Presione Intro. Herramientas para desarrolladores evalúa la instrucción y muestra 6, que es el resultado que esperas que produzca la demostración.

k) Solucionar el problema:

1. Haz clic en Reanudar ejecución de la secuencia de comandos .
2. En el editor de código, reemplaza la línea, `var sum = addend1 + addend2`, por `var sum = parseInt(addend1) + parseInt(addend2)`.
3. Presiona Comando + S (Mac) o Ctrl + S (Windows y Linux) para guardar los cambios.
4. Haz clic en Desactivar puntos de interrupción . Su color cambiará a azul para indicar que está activo. Mientras esté configurado este parámetro, Herramientas para desarrolladores ignorará los puntos de interrupción que hayas establecido.
5. Prueba la demostración con diferentes valores. Ahora los cálculos se realizan de manera correcta.
6. ¿Qué opinas de estos métodos de depuración?
7. ¿Cuál resulta para ti la importancia de los breakpoint/puntos de interrupción?



Reto 4 (at home)

Hacer un análisis de *profile.html* e *index.html* empleando las herramientas de desarrollador vistas:

1. Actualizar el árbol DOM de la página de *profile.html* y cree el árbol DOM de *index.html*. Recuerde incluir estos diagramas en el directorio “docs” de su proyecto.
2. Crear el árbol CSSOM de los estilos más relevantes de la página de listado. Colocar éste diagrama en el directorio “docs”
3. Dibujar el árbol de renderización correspondiente a la página de listado. Colocar éste diagrama en el directorio “docs”

Note que no es necesario crear los árboles CSSOM o de renderización para la página de perfil.

4. Seleccionar una herramienta estudiada de su preferencia (e.g. Accesibilidad, Performance, Almacenamiento, Red, etc.) y ejecute una prueba (test) utilizando dicha herramienta. Tome una captura de pantalla con los resultados de su prueba.
5. Redactar en un párrafo qué valor aporta la herramienta usada, incluya en su reflexión sobre la herramienta si ésta le permitió visualizar algún área de mejora para su aplicación.
6. Coloque los insumos anteriores (diagramas, capturas de pantallas y reflexión sobre la herramienta) en un archivo pdf, membretado con sus datos, con el nombre “laboratorio4<Nombre Apellido>”, colóquelo en el aula virtual.

Al terminar, colocar en el reto del Aula Virtual el url del repositorio de GitHub que uso para hacer este ejercicio, una imagen del historial de Git Graph que muestra su Visual Code y el archivo pdf creado.